

Program-09

Title: Organizing Kubernetes Pods Using Labels and Namespaces

Objective

To create logically isolated environments within a Kubernetes cluster by using Namespaces, and to categorize and manage resources efficiently through Labels for improved filtering and administration.

1. Environment Preparation

The project setup begins by creating two essential configuration files to define the cluster structure and workloads:

namespaces.yaml – Used to specify and create isolated Kubernetes namespaces.

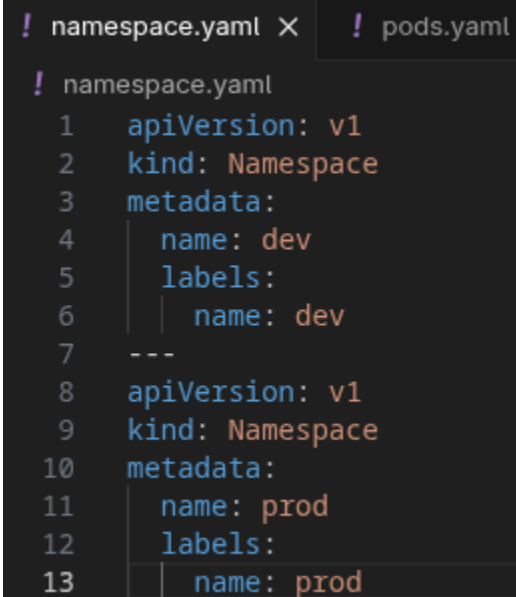
pods.yaml – Used to define application Pods along with relevant metadata such as labels.

2. Implementing Logical Isolation with Namespaces

Namespaces enable the division of a single Kubernetes cluster into multiple virtual environments. In this lab exercise, the following namespaces were created:

dev – Dedicated to development-related workloads.

prod – Reserved for production-level workloads



```
! namespace.yaml X ! pods.yaml
! namespace.yaml
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: dev
5    labels:
6      name: dev
7  ---
8  apiVersion: v1
9  kind: Namespace
10 metadata:
11   name: prod
12   labels:
13     name: prod
```

3. Resource Identification Using Labels

Labels are user-defined key–value pairs that are assigned to Kubernetes resources such as Pods. They provide meaningful identifiers that help categorize, organize, and manage resources efficiently.

In this setup, the following metadata labels are defined in the pods.yaml file:

app: frontend or app: backend – Distinguishes the functional role of each Pod within the application architecture.

env: dev or env: prod – Specifies the environment to which the Pod belongs, such as development or production.

! namespace.yaml

! pods.yaml X

! pods.yaml

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: frontend-pod
5    namespace: dev
6    labels:
7      app: frontend
8  spec:
9    containers:
10   - name: frontend
11     image: nginx:latest
12     ports:
13       - containerPort: 80
14   ---
15  apiVersion: v1
16  kind: Pod
17  metadata:
18    name: backend-pod
19    namespace: dev
20    labels:
21      app: backend
22  spec:
23    containers:
24   - name: backend
25     image: nginx:latest
26     ports:
27       - containerPort: 80
28   ---
29  apiVersion: v1
30  kind: Pod
31  metadata:
32    name: frontend-pod
33    namespace: prod
34    labels:
35      app: frontend
36  spec:
37    containers:
38   - name: frontend
39     image: nginx:latest
40     ports:
41       - containerPort: 80
42   ---
43  apiVersion: v1
44  kind: Pod
45  metadata:
46    name: backend-pod
47    namespace: prod
48    labels:
49      app: backend
50  spec:
51    containers:
52   - name: backend
53     image: nginx:latest
54     ports:
55       - containerPort: 80
```

4. Applying the Deployment Configurations

Once the configuration files are prepared, they are applied to the Kubernetes cluster using the `kubectl apply` command.

Namespace Creation:

Executing `kubectl apply -f namespaces.yaml` creates the required namespaces, namely `dev` and `prod`, within the cluster.

Pod Deployment:

Running `kubectl apply -f pods.yaml` deploys the defined Pods (for example, `frontend-pod` and `backend-pod`) into their designated namespaces based on the configuration.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl apply -f ./namespace.yaml
namespace/dev created
namespace/prod created
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl apply -f ./pods.yaml
pod/frontend-pod created
pod/backend-pod created
pod/frontend-pod created
pod/backend-pod created
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6f9664446b-2599x  1/1     Running   2 (4h24m ago)  4h34m
nginx-deployment-6f9664446b-lbr2k  1/1     Running   2 (4h24m ago)  4h34m
○ rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$
```

5. Advanced Resource Filtering

Once resources are deployed, you can use Selectors (the `-l` flag) and Namespace flags (the `-n` flag) to filter the output.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl get pods -n dev
NAME            READY   STATUS    RESTARTS   AGE
backend-pod     1/1     Running   0           2m25s
frontend-pod    1/1     Running   0           2m25s
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl get pods -n prod
NAME            READY   STATUS    RESTARTS   AGE
backend-pod     1/1     Running   0           2m30s
frontend-pod    1/1     Running   0           2m30s
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl get pods -n dev -l app=frontend
NAME            READY   STATUS    RESTARTS   AGE
frontend-pod    1/1     Running   0           3m10s
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$ kubectl get pods -n dev -l app=backend
NAME            READY   STATUS    RESTARTS   AGE
backend-pod     1/1     Running   0           3m16s
○ rv24mc081_r_mahesh_babu@fedora:~/Downloads/PRG6$
```